

Guia de Segurança — Pré-Produção

Checklist do que precisa ser feito antes de colocar o sistema no ar para um cliente real. Cada item traz a exigência técnica e uma explicação em linguagem simples.

✓ **O que já está pronto (no código):** autenticação própria com token de curta duração e renovação segura; senha protegida por hash; isolamento entre clientes (cada empresa só vê os próprios dados); proteção contra as principais formas de ataque (injeção de SQL, XSS, CSRF, acesso indevido a arquivos); o token de acesso fica fora do alcance de scripts (BFF); cabeçalhos de segurança; limite de requisições; e verificação automática de dependências no CI. **Este guia cobre só o que falta — e o que falta é de infraestrutura/operação, não de programação.**

1. Segredos

"Segredos" são as senhas e chaves que o sistema usa por baixo dos panos (para assinar logins, acessar o banco, etc.). Se vazarem, alguém pode se passar pelo sistema.

Chave de assinatura de login (`AUTH_JWT_SECRET`) única por ambiente **ALTA**

Aleatória de 64+ caracteres (`openssl rand -base64 48`); nunca reutilizar entre ambientes; nunca versionar.

O que é: uma senha-mestra que o sistema usa para "carimbar" os crachás de login (os tokens). **O que fazer:** gerar uma sequência aleatória longa e diferente para cada ambiente (teste, produção) e guardá-la nas configurações do servidor — nunca dentro do código. **Por quê:** quem tiver essa chave consegue fabricar crachás falsos e entrar como qualquer usuário. *(No ambiente local já geramos uma real.)*

Usuário do banco de dados com permissão mínima **ALTA**

Não usar `sa` ; usuário com privilégio restrito, rotacionável.

O que é: a "conta" que o sistema usa para falar com o banco de dados. **O que fazer:** criar uma conta específica que só pode fazer o necessário (ler/gravar nas tabelas do app), em vez do administrador-geral (`sa`). **Por quê:** se essa conta vazar, o estrago fica limitado — o invasor não vira dono do banco inteiro.

Demais chaves (storage, etc.) em cofre de segredos **MÉDIA**

Chaves de R2/S3 e afins via cofre/variáveis do orquestrador — nunca no repositório nem na imagem.

O que é: chaves de acesso a outros serviços (por ex. onde os anexos ficam guardados). **O que fazer:** guardar todas num "cofre" de segredos do servidor (ou variáveis de ambiente), fora do código. **Por quê:** código costuma ser copiado/compartilhado; segredo dentro dele vaza fácil.

2. Transporte (a conexão)

"Transporte" é como os dados viajam entre o navegador do usuário e o servidor. O objetivo é que ninguém no caminho consiga ler ou adulterar.

HTTPS/TLS ligado, com redirecionamento de HTTP → HTTPS ALTA

TLS terminado no reverse proxy; o `Strict-Transport-Security` só tem efeito sob HTTPS.

O que é: o "cadeado" do navegador — a criptografia da conexão. **O que fazer:** instalar um certificado (ex. Let's Encrypt) no servidor de entrada e forçar que todo acesso use `https://` (quem entrar por `http://` é redirecionado). **Por quê:** sem isso, senhas e dados trafegam em texto aberto e podem ser lidos numa rede Wi-Fi, por exemplo. Várias proteções que já programamos só "ligam" sob HTTPS.

Cookies marcados como `secure` em produção MÉDIA

Já ativado automaticamente quando `NODE_ENV=production`.

O que é: uma marca que impede o crachá de login de viajar por conexão sem cadeado. **O que fazer:** basicamente nada além de rodar em modo produção — **já está programado**. Só confirmar que o ambiente está em produção.

3. Dados

Proteção das informações guardadas (banco de dados e arquivos): contra perda e contra acesso indevido.

Backup automático + teste de restauração ALTA

Backup do banco automatizado; testar a restauração.

O que é: cópias de segurança dos dados. **O que fazer:** agendar cópias automáticas do banco (diárias, por ex.) e — importante — **testar de verdade** se consegue restaurar a partir delas. **Por quê:** backup que nunca foi testado costuma falhar justo na hora da emergência (defeito de disco, ransomware, erro humano).

Criptografia dos dados em repouso MÉDIA

Encryption at rest do banco e do storage de anexos.

O que é: deixar os dados criptografados também quando estão "parados" no disco. **O que fazer:** ativar a criptografia de disco/volume do banco e do local dos anexos (a maioria dos servidores e nuvens oferece isso com um clique). **Por quê:** se alguém roubar o disco/HD ou uma cópia dele, não consegue ler nada.

Política de retenção/limpeza de dados pessoais (LGPD) MÉDIA

Retenção/limpeza de PII de parceiros e de logs de auditoria.

O que é: regras de por quanto tempo guardar dados pessoais e quando apagá-los. **O que fazer:** definir prazos (ex.: apagar/anonimizar dados de parceiros inativos após X anos) para cumprir a LGPD. **Por quê:** guardar dado pessoal sem necessidade é risco legal e de vazamento.

4. Escala (só se houver mais de um servidor)

Vale apenas se, no futuro, o sistema rodar em vários servidores ao mesmo tempo. Para um único servidor (caso on-premise típico), pode ignorar.

Limite de requisições em armazenamento compartilhado (Redis) CONDICIONAL

O throttle atual é in-memory (ok para VM única); migrar para Redis em multi-nó.

O que é: a proteção que barra excesso de tentativas (ex.: ataque de força bruta no login) hoje "conta" na memória de um servidor só. **O que fazer:** só se houver vários servidores — usar um contador central (Redis) para todos compartilharem. **Por quê:** com vários servidores e contadores separados, um atacante teria o limite multiplicado. *Num servidor só, o que já existe basta.*

5. Operação (o dia a dia)

Como acompanhar e reagir a problemas depois que o sistema está no ar.

Monitoramento e alertas de anomalias de login MÉDIA

Usar `LoginEvent` (falhas, bloqueios) e hits de throttle.

O que é: um "alarme" que avisa quando algo estranho acontece (muitas tentativas de login falhas, contas sendo bloqueadas). **O que fazer:** o sistema já **registra** esses eventos; falta ligar um painel/alerta que avise a equipe (e-mail, mensagem) quando os números fugirem do normal. **Por quê:** quanto antes você percebe um ataque, menor o estrago.

Rodar o scanner de dependências em todo push/PR MÉDIA

CI já criado (`.github/workflows/ci.yml`): `build + testes + npm audit`.

O que é: uma verificação automática que checa se alguma "peça" de terceiros usada pelo sistema tem falha de segurança conhecida. **O que fazer: já está pronto** — o robô roda a cada alteração e trava se aparecer falha grave. Só manter o hábito de revisar quando ele apontar algo.

Teste de invasão (pentest) formal antes do lançamento **ALTA**

A auditoria interna cobriu as classes principais, mas não substitui teste independente.

O que é: contratar um especialista (externo) para tentar "invadir" o sistema de propósito e reportar as brechas. **O que fazer:** antes do lançamento oficial para clientes, contratar um pentest. **Por quê:** nós fizemos uma auditoria interna forte, mas um olhar independente sempre acha coisas que quem construiu não vê. É o padrão do mercado para software que lida com contratos e dados de empresas.

Nxt · Guia de Segurança Pré-Produção · gerado a partir de `SECURITY.md` · uso interno.

Prioridades: **ALTA** = obrigatório antes de qualquer cliente · **MÉDIA** = fazer na implantação · **CONDICIONAL** = só em cenário específico.